

Orgalyst: a conversational analysis system for organoid bright-field images

A bench scientist says one sentence and, minutes later, has an analysis report that can go into the lab record and still be verified three months later; every number in it can be recomputed from the command line without any large language model.

Category: End-to-End System

Demo video	≤ 5 min, public: <code><video link, to be added></code>
Code	https://github.com/xiaoyanLi629/orgalyst (public, reproducible, MIT)
Technical report	PDF: https://github.com/xiaoyanLi629/orgalyst/blob/main/docs/technical_report_en.pdf (English, 18 pages of main text plus appendix) · https://github.com/xiaoyanLi629/orgalyst/blob/main/docs/technical_report.pdf (Chinese); HTML editions in the same folder
Weights	https://huggingface.co/XiaoyanLi/orgalyst-weights
Data	OrgLine (Zenodo 16355179, CC-BY-4.0); no private data, no additional annotation
Team	Xiaoyan Li (AI / computer vision / LLM applications, project lead); Liwen Xu and Xuzheng Fu (biology: organoid culture and morphological interpretation, responsible for requirements and review of results); Cuicui Jiang (AI: large language models, agent architecture and orchestration)

1 The problem

Organoids are miniature tissues grown from stem cells in a three-dimensional environment; organ-on-a-chip devices place such tissues into microfluidic hardware that mimics physiological conditions. In both cases the cheapest, least invasive and most frequent way to observe them is the bright-field microscope: take a picture every day or every few hours and see how they are growing. The analysis these pictures need is very fixed: count how many organoids are in an image, measure the size and shape of each, compare treatment groups, connect measurements of the same organoid over time into a growth curve, and write it up as a report that can go into the lab record and be checked by someone else.

The bottleneck is not the algorithm. Cellpose, OrganoID and OrgaSegment already segment organoids well, and the OrgLine dataset has unified eight public sources into one format. What actually stops bench scientists are three

things. The tools are for people who write code: an environment must be installed, a model chosen and a diameter parameter tuned, and the defaults fail when organoids in one image differ tenfold in size. Segmentation is only the first step; the measuring, statistics, plotting and report writing that follow are reinvented in every lab, inconsistently. And nothing is recorded: three months later a reviewer asks which model version and which parameters produced this area, and most people cannot answer.

Orgalyst targets biologists who run organoid or organ-on-a-chip experiments and do not write code. We set three criteria for success: a complete analysis report from natural language alone; every number recomputable from the command line without any language model, and consistent with the expert annotation shipped with a public dataset; and a system that says when it is unsure instead of returning a wrong number that looks precise.

2 What the system is

Two layers. The **toolkit** `orgalyst` is an ordinary Python package with the command-line entry point `python -m orgalyst analyze | count | track | report`; it needs no language model, and every run leaves a directory with `manifest.json` (md5 of each input, model identifier and weight hash, all parameters, software versions, GPU, step log), per-instance feature tables, summaries, QC and comparison tables, masks, overlays and a single-file `report.html`. The **assistant** is built on the Claude Agent SDK; the toolkit is exposed through one MCP server as six tools (list models, analyse, count, compare groups, growth curves, review runs), and the assistant interprets the request, picks the organ weights, passes parameters, checks and explains results. It never looks at pixels or computes numbers itself.

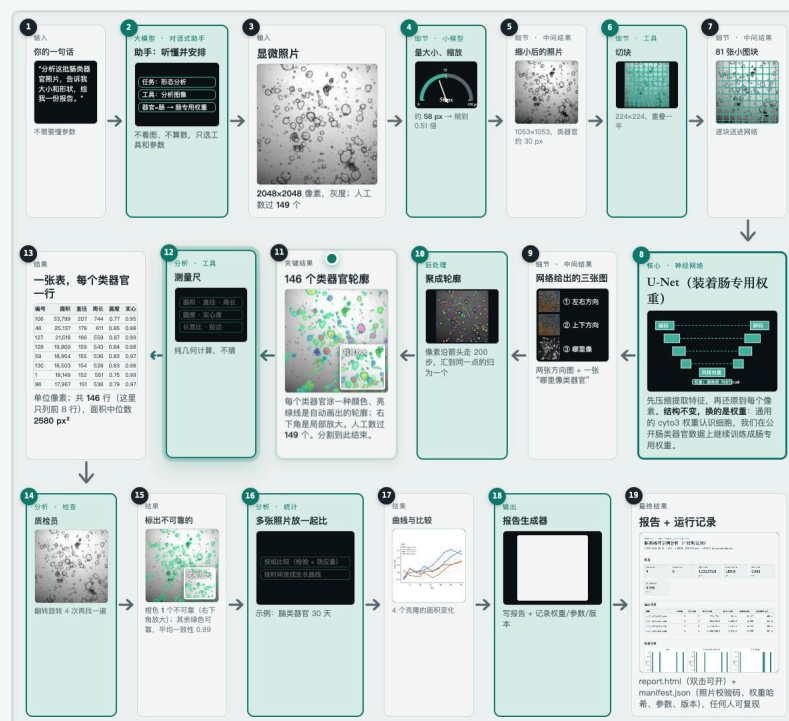


Figure 1 · The complete journey of one intestinal organoid image. White cards are data, green boxes are models or tools, large cards are the key steps: from the user's sentence to tool selection, size estimation, rescaling, tiling, the U-Net loaded with intestine-specific weights, the three network outputs, tracking into contours, 146 organoid outlines, the measurement table, QC, statistics, the report generator, and finally the report plus the run record. Labels in Chinese; interactive version at `docs/orgalyst_scene.html` in the repository.

2.1 What the toolkit does

Segmentation. Cellpose (a U-Net regressing a flow field; pixels follow the flow and converge into instances), but not with its generalist weights: we continued training cyto3 on OrgLine for 200 epochs separately for brain, intestine, pancreatic cancer (pdac) and colon, giving four organ-specific weight sets, and chose a diameter strategy per organ: colon uses the training diameter stored in the weights, intestine and pdac estimate the diameter per image with the size model, brain runs two passes (the second at the median diameter of what the first pass found). Cellpose's most common failure on organoids is scale: the generalist model estimates 30 pixels for brain organoids 400 pixels wide, giving zero-shot AP50 of 0.005. The organ weights remember the scale, and two-pass inference handles brain organoids that grow almost fivefold in area from day 2 to day 30. Unknown organs fall back to the generalist model with a "lower accuracy" notice in the report.

Counting. When the user only asks how many, segmentation is wasteful; a YOLO11m detector trained jointly on three organs answers in well under a second. Its counting threshold is not the default 0.25 but the value with the lowest counting error on each organ's validation set (intestine 0.50, brain 0.45, lung 0.40).

Morphometry, QC, statistics, growth curves. Measurement gives area, equivalent diameter, perimeter, circularity, solidity, aspect ratio and a border flag, with micrometre columns when a pixel size is given; incomplete border-touching instances are excluded by default and counted in the report. QC works at two levels: image-level sharpness, uneven illumination and saturation; instance-level test-time augmentation with flips and rotations, flagging an instance as "low confidence" when it is recovered in fewer than half of the transforms, drawn in orange in the report. Group comparison uses Mann-Whitney or Kruskal-Wallis with Cliff's delta and Holm correction; growth curves connect per-image median area over individuals, time points and groups and give fold change relative to the first time point.

2.2 How the assistant works

Tools alone are not enough. When the assistant should ask what, run what first, how to check and how to report is written as a skill (`agent/skillpack/skills/organoid-analysis`) that activates whenever the user mentions organoid image analysis. It prescribes four steps: confirm three prerequisites (organ, pixel size, and whether counts, morphology or a comparison is wanted; ask for what is missing without blocking simple tasks); choose tools from them (counting goes to detection, everything else to segmentation, QC for small batches or when reliability matters, statistics when group or time-point information exists); check the returned values before answering (an outlying per-image count triggers a look at the overlay; border exclusions and low-confidence counts must appear in the answer); report in a fixed six-part format. The skill also lists prohibitions: no micrometre figures without a pixel size, no recomputing in Python what a tool already reported, no re-running the same batch.

How the user asks	What the assistant does	What the user gets
"How many organoids are in these images?"	Detector counts per image at the calibrated threshold	Per-image count table, images with boxes
"Analyse these intestinal organoid images, tell me size and shape"	Organ weights, segmentation, measurement, report	Total, per-image table, medians and IQR of area / diameter / circularity, report.html
"Pixel size is 3.16 μm , give areas in μm^2 "	Passes the calibration to the tool	Results in micrometres; without a calibration it says "pixels only"
"Which segmentations are unreliable?"	Two-level QC	Image-quality metrics, low-confidence instances in orange
"Is there a difference in area between control and treated?"	Tests and effect sizes from a group file	Medians per group, p value, Cliff's delta, box plot, plain-language conclusion
"How much did it grow from day 2 to day 30?"	Growth curves over individuals and time points	First and last values, fold change, mean curve per group
"Which model and parameters? How do I reproduce this?"	Reads the run manifest	Path of manifest.json and the hashes, parameters and versions it records
"Analyse the images in /data/x" (empty / organ unstated)	Reports no images found; or uses the generalist model with a caveat	One clear statement instead of a plausible-looking number



Figure 2 · A real analysis in the web interface (4 intestinal organoid images; interface language Chinese).

Centre: the assistant's answer with prerequisites, per-image counts, overall morphology, output paths and suggestions. Top right: the confirmation mode of this conversation; right column: the task list the assistant maintains and the files produced; bottom: context usage and spending. The same backend has a terminal edition for scripted evaluation.

The end of the assistant's answer to that task reads as follows (excerpt, translated). It states the units, says it did not invent a pixel size, suggests checking the overlays first, and states plainly that 201 is a model output without an accuracy guarantee:

"Note: all sizes are in pixels, not micrometres. You did not provide pixel_size_um and I have not invented a calibration. ... I suggest looking through the 4 overlay images in overlays/ first. This run used the intestine-specific model, but without manual annotation as a gold standard, 201 is still just a model output with no accuracy guarantee."

3 Results

All five experiment groups were run on the OrgLine test sets; the segmentation metric is instance-level AP50 (a prediction matches a ground-truth instance at IoU ≥ 0.5).

E1 segmentation AP50	cyto3 zero-shot	Per-organ fine-tuned	Fine-tuned + per-ima
brain (240)	0.005	0.925	0.967
intestine (12)	0.544	0.773	0.812
colon (10)	0.405	0.815	0.814
pdac (20)	0.419	0.564	0.591

Three conclusions: zero-shot cyto3 recognises organoids but over-segments badly, and fine-tuning fixes both scale and boundaries; the diameter strategies pay off (brain 0.925 $\rightarrow$ 0.967, intestine 0.773 $\rightarrow$ 0.812); pdac remains weak, with more than 500 small instances per test image from a different instrument and recall of about one half. Leave-one-organ-out experiments show limited cross-organ generalisation (0.15 to 0.47 on unseen organs), so organ-specific weights are necessary; fine-tuned Cellpose-SAM matches but does not beat fine-tuned cyto3 (pdac 0.557, intestine 0.780) with 50 $\times$ larger weights and twice the inference time, so it is kept only as a comparison.

E2 counting (joint YOLO11m)	mAP50	Calibrated threshold	Count MAPE (0.25 $\rightarrow$ calibr)
intestine (423)	0.948	0.5	0.262 $\rightarrow$ 0.207
brain (280)	0.995	0.45	0.018 $\rightarrow$ 0.004
lung (602)	0.936	0.4	0.235 $\rightarrow$ 0.168

E3 agreement with expert annotation (brain, 240 images, automatic area vs the manual area in the original dataset): detected 237 / missed 3, Spearman 0.963, median relative error 1.9%, 93% of images within 10%. **E4 quality control** (intestine, colon and pdac test sets): the share of wrong instances (IoU < 0.5) among those flagged low-confidence is 3.5 to 6.8 times that of the rest, but recall is only 0.07 to 0.13, so the report presents the flag as a hint rather than a filter.

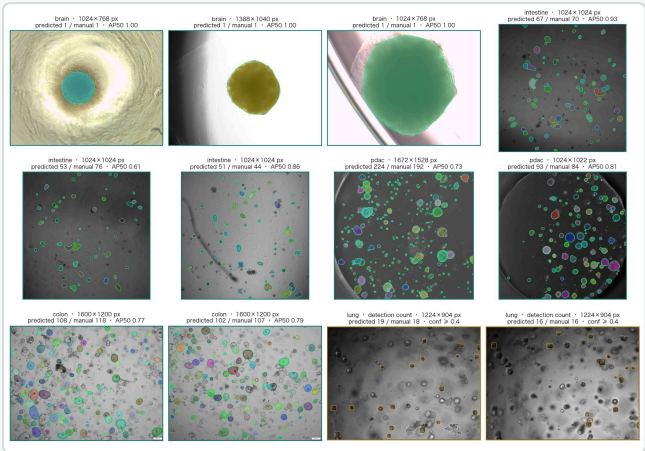


Figure 3 · The default pipeline on 12 unselected test images. Segmentation on brain, intestine, pdac and colon, detection counting on lung; titles give predicted count, manual count and per-image AP50.

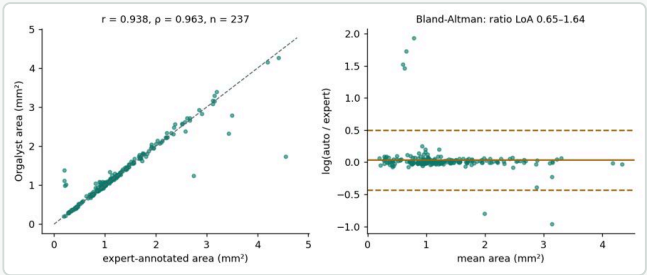


Figure 4 · E3 scatter and Bland-Altman plots of automatic vs expert area. Residual failures are the smallest day-2 organoids (4 over-estimated) and one of the largest day-30 ones.

E5 end-to-end agent benchmark. Twelve natural-language tasks, each with an automatically judged success criterion: numbers in the answer must lie within 5 to 10% of the command-line reference values, the agreed output files must exist, and key wording must be present. With the Claude (claude-opus-5) backend 12/12 tasks passed, at 113 s and 10.5 tool calls per task on average (1.5 of them Orgalyst tools), 5.63 USD in total.

Task	Content	Time (s)	Tool calls	of which Orgalyst	Verdict
t01	single image: count and size	44	2	1	pass
t02	batch analysis with report	64	3	1	pass
t08	follow-up: metric definitions	65	5	0	pass
t03	analysis with pixel size	49	2	1	pass
t10	follow-up: reproducibility	28	2	0	pass
t04	count only (detection)	40	1	1	pass
t05	group comparison	309	31	5	pass
t06	growth curve	458	46	3	pass
t07	quality control	154	18	1	pass
t09	organ not specified	62	3	2	pass
t11	empty folder	24	2	0	pass
t12	mixed-organ batch	65	11	3	pass

4 Reproducibility

The repository offers two paths. The **command-line path** needs no language model: `pip install -r requirements.txt`, `bash scripts/download_weights.sh` (about 200 MB), then `python -m orgalyst analyze --organ intestine --images DIR --out runs` produces a report for any folder of bright-field images. The script order and expected wall time for reproducing every table (data preparation 15 min, E1 fine-tuning 2.5 h, cross-organ 3 h, E2 detection 4.5 h, E3 10 min, E5 25 min) are in the README and section 10 of the technical report; the repository already contains all result files we produced, and a Dockerfile covers this path. The **agent path** uses the Claude API by default (reviewers need their own key) and can be pointed through `ANTHROPIC_BASE_URL` at an open-model gateway that speaks the Anthropic interface; `agent/` holds the complete assistant code with no accounts or secrets.

5 Limitations

Recall on pancreatic cancer organoids is only about one half and needs more in-domain data or multi-scale inference for small instances; only the brain dataset has a pixel calibration, so other organs are reported in pixels; the intestine

(12) and colon (10) test sets are small and their AP50 confidence intervals wide; the agent path was evaluated on one backend and twelve tasks cannot cover every way real users phrase requests; all data come from published datasets and none from organ-on-a-chip time series. Section 8 of the technical report expands on each.

6 Impact and next steps

For bench scientists, the path from image to report shrinks from half a day to minutes and the report is verifiable; for method developers, adding an organ takes one weight file and one line of configuration. The three most valuable next steps: link detection boxes over time for per-individual tracking of organoids fixed in place on a chip; fit dose-response curves to morphological metrics; and attach the assistant to the scheduling interface of an imaging system so that "analyse after imaging, alert on anomalies" closes the loop.

Claude Code was used as a coding assistant during development; all code, documents and experiment scripts were reviewed by the authors. Code MIT; OrgLine CC-BY-4.0 (not redistributed); weights derive from Cellpose (BSD-3) and Ultralytics YOLO11 (AGPL-3.0).